Categorical Framework for Visualization Software Architecture

Hannah Aizenman *
Computer Science, The Graduate Center, CUNY
Mikael Vejdemo-Johansson †
Mathematics, CUNY College of Staten Island
Thomas Caswell ‡
National Synchrotron Light Source II, Brookhaven National Lab
Michael Grossberg §
Computer Science, City College of New York, CUNY

Abstract

The contract data visualization tools make with their users is that a chart is a faithful and accurate visual representation of the numbers it is made from. Motivated by wanting to make tools that are more flexible while still rigorously structure preserving, we propose a methodology for fully specifying arbitrary data to visualization mappings in a manner that translates to pseudocode. We propose that fiber bundles provide a uniform interface for describing a variety of underlying data - tables, images, networks, etc. - in a manner that independently encodes the mathematical structure of the continuity and the fields of the dataset. Modeling the data structures that store the datasets as sheaves provides a framework for specifying visualization methods that are designed to work regardless of how the dataset is stored - whether the data is on disk, distributed, or on demand. Specifying the visualization library components as natural transforms of sheaves means that the constraints that the component must satisfy to be structure preserving can be specified as the set of morphisms on the data and graphic sheaves, including the structure on the topology and fields of the data. Using category theory to formally express how visual elements are constructed means we can translate those expectations into pseudocode, which can then be used to build tools that enforce the expectation that a visualization tool is faithfully translating between numbers and charts.

Index Terms: visualization software design, visualization theory

1 Introduction

Visualization design guidelines, generally, describe how to choose data $\rightarrow$ graphic encodings that preserve the structure of the data; Loosely, preserving structure means that the properties of the data and how the points are connected to each other should be inferable from the graphic such that a graphic $\rightarrow$ data mapping can be made. For example, values read off a bar chart have to be equivalent to the values used to construct that chart. Therefore, a visualization tool is structure preserving when it preserves the bidirectional mapping data $\leftrightarrow$ graphic. Visualization tools try to reliably guarantee the data to graphic side of the equation, but implementation is sometimes imperfect. For example in Matplotlib there are different APIs for accessing each type of data (e.g. line and image) and this leads to changes to the data not always being applied consistently across plotting methods. Wanting to regularize the units (which manages plotting physical units, such as dates) behavior motivated the need for a uniform data interface and framework for expressing what correct transforms look like across the plotting methods. We propose that structures from category theory and algebraic topology can be used as a sort of pseudocode for uniformly specifying the behaviors and properties of data and graphic and rigorously expressing the structure preserving constraints on maps between them. By design, this pseudocode translates to strong programmatic types and a functional design framework, which means these components can be built as a standalone library and integrated into existing libraries. This psuedocode also has the constraints baked into the definitions of the structures, meaning that developers can use the structure to express the constraints and trust that the math has been worked out. The contributions of this paper are a framework that provides a uniform interface for arbitrary datasets in arbitrary containers, uses category theory as the language for expressing what structure needs to be preserved across mapping and provides guidance for the construction and testing of structure preserving visualization library components.

*email:haizenman@gradcenter.cuny.edu
†e-mail: mvj@math.csi.cuny.edu
‡e-mail: tcaswell@bnl.gov
§e-mail: mgrossberg@ccny.cuny.edu

2 Related Work

Our model generalizes previous mathematically derived frameworks by allowing arbitrary structures, explicitly incorporating topology, and by providing an explicit framework for translating the theoretical ideas into buildable components. Bertin [6] codified the idea that properties of data and graphics should match, using the Steven's measurement scales [47] as a case study. Mackinlay and Wilkinson translate algebras on data to grammars for specifying graphics to develop frameworks for designing structure preserving visualization tools [31, 33, 55], while our framework uses the same abstractions for data and graphics. Vickers et al [52] apply category theory to visualization semantics, while our focus is structural. Bartonicek built a tool based on the algebraic properties of summary statistics [2]; our work provides a more general framework for codifying those properties and proposes that sheaves are a better abstraction for reasoning about connectivity constraints. Similarly to Kindlemann and Scheidegger's algebraic visualization design (AVD) framework [25] — but with a focus on software architecture rather than visualization design — we present a uniform abstract data representation layer for ensuring that the visualization should not change if the data representation changes, define the conditions under which data is mapped unambiguously to visual encodings [59], and provide a methodology for verifying that changes in data should correspond to changes in the visualization.

2.1 Structure Preservation In Software

The visual algorithms implemented in building block libraries [1] are generally assumed to preserve the underlying continuity of their input data, as described in taxonomies of visualization algorithms by Chi [11] and by Troy and Möller [51]. In libraries such as Matplotlib [24] and D3 [8], assumptions about connectivity are embedded in the interfaces such that the API is inconsistent across plot types. For example in Matplotlib methods for updating data and parameters for controlling aesthetics differ between (1D) line based plotting methods and (0D) marker based methods. While libraries like VTK [17, 13] and Bokeh [7] provide a way to express the connectivity of the data, and therefore can embed that information in their visual algorithms, their charting APIs are similar to the continuity dependent APIs of other

[1] a term introduced by Wongsuphasawat to describe libraries that provide modular components for building elements of a visualization [57, 56]

building block libraries. In contrast, domain specific libraries are designed with the assumption of continuities that are common in the domain [20], and therefore can somewhat restrict their API to choices that are appropriate for that domain. For example, a tabular structure of discrete rows, as illustrated in Figure 1, is assumed by Tableau [48, 16, 32] and excel charts, which try to suggest structure preserving visualizations, and ggplot [53] and vega [41], which leave the choice to the user. Image libraries such as Napari [43] and ImageJ [42] and the ImagePlot [49] art plugin assume that the input is a 2D continuous image. Networking libraries such as gephi [3] and networkxx̌citehagbergExploringNetworkStructure2008 assume a graph-like structure. By assuming the structure of their data, these domain specific libraries can provide interfaces that are easier to navigate due to more similar specification notation [27] because the visual algorithms can share assumptions about the underlying data. We propose that the cohesiveness of domain specific library APIs is obtainable via a uniform data model, while the expressivity [31] of building block libraries can be preserved by using category theory specify functional components [21, 22] that are easily composable due to consistent software interfaces [54, 58].

3 Uniform Abstraction for Data & Graphics

Generally, preserving structure means that a visualization is expected to preserve the field properties and connectivity of the corresponding dataset. We use **field**[2] to denote the set of values of the same type, e.g. date, location, stations, and precipitation in a weather station dataset and **connectivity** to denote how elements are arranged in a dataset [55], e.g. as 0D (•) points, along a 1D (–) line, in a 2D (■) surface, as a 3D trajectory, a volumetric cube, or as a network. We propose a mathematical abstraction of the data input and pre-rendered graphic output that provides a uniform highly generalizable interface for describing topology and fields; expresses how to verify that data connectivity is preserved on subset, distributed, and streaming data representations; and formalizes the expectation of a correspondence between data and visual elements. Using these abstractions allows us to express the structure of data and graphics using the type signature `dataset` : `connectivity` → `fields`, which means visualization algorithms can check the data input against a signature to ensure that the assumptions of the data match the assumptions of the algorithm. This typing system also extends to pre-rendered graphic output, allowing us to develop the structure preservation framework that ensures that output fields are equivalent to the input fields and that the connectivity of the output graphic matches the connectivity of the input.

3.1 Abstract Data Representation

We model data using a mathematical representation of data that can encode topological properties, field types, and data values in a uniform manner using a structure from algebraic topology called a fiber bundle. We extend Butler's proposal of using bundles as an abstraction for visualization data [9, 10] by incorporating Spivak's methodology for encoding named data types from his fiber bundle representation of relational databases [45, 46].

Encoding the data as fiber bundle provides a uniform interface for individually specifying the connectivity and field properties and the relationship between these spaces `dataset` : `connectivity`→`field` for datasets with arbitrary structures. This is because a **fiber bundle** $F \hookrightarrow E \overset{\pi}{\twoheadrightarrow} K$ is a structure with topological spaces E, F, K and bundle projection map $\pi : E \to K$ [44](Supplement section 4) and we propose that the total space E of a bundle can encode the mathematical space in which a dataset is embedded, the base space K can encode the connectivity of the dataset, the fiber space F can encode the field properties of the dataset, and that the datasets can be encoded as section functions τ from the base space to the fiber space.

Fiber bundle spaces can also be partitioned into sub bundles, called local trivializations, with transition maps to glue the local trivializations back together. For example, we can unwrap a sphere into a plane to show that a continuous map exists between a segment of the globe and a corresponding choropleth, thereby addressing Wilkinson's concern that connectivity cannot be tested in a generalizable way because continuous maps do not always exist, specifically between a planar geographic maps and the globe [55]. The subsets of the fiber bundle and the transition maps between these subsets are encoded in an atlas [14] and the notion of an atlas can be incorporated into the data container, as discussed in subsection 3.2.

3.1.1 Connectivity: Base Space K

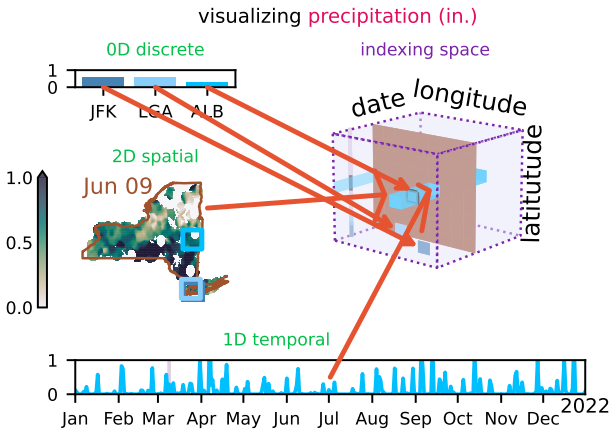


Figure 1: This weather station data has multiple embedded continuities - points at each time and position, timeseries at each position, and maps at each time. The corresponding visualizations - bar chart, timeseries, and map - each preserve the continuity of the subset of the data they visualize by not introducing or leaving out values and preserving the relative positioning of continuous values.

The base space K can be thought of as an indexing set for the data – where tabular data may have a discrete indexing set, time-series are indexed over one-dimensional time, and spatial fields are indexed over d-dimensional space, as illustrated in Figure 1 – because the index is encoding the connectivity. Framing this in the language of topology provides us with an abstraction of our data that can be unified over data representation (in memory, distributed, streaming) and over different connectivity types (discrete, d-dimensional). There are many formal definitions of a topology; we choose to work with the neighborhood space axiomatization (Supplement subsection 1.3) because it is most analogous to the data access model of index (element) in subset (neighborhood) of all indices (base space).

Definition 3.1. We define the base space to be an indexing space K with indices k and a neighborhood topology. A neighborhood of k has to contain k, can grow, or shrink, and every neighborhood contains smaller neighborhoods of points adjacent to k.

In the indexing cube in Figure 1, the brown surface and blue rectangle cube are both neighborhoods of the index for the measurement in Albany on June 09. The blue rectangle is also a neighborhood of the index for the measurement in Albany on March 09. The indexing cube is a neighborhood for both of these indices. Using Munzner's key-value based semantic indexing [36], the data in Figure 1 could be subset into sets of timeseries where

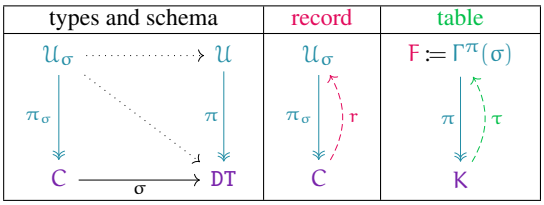
[2] In this paper, color in definitions, equations, and visualizations is used to group conceptually related terms [19], e.g. field and fiber.

the key would be `station`, or subset into maps where the key would be `date`, or subset into station records where the keys are (`date`, `latitude`, `longitude`). Using a connectivity model expresses how different labeling schemes can refer to the same indices, e.g. how 0-360 and 180E-180W are two ways of labeling longitude or how (date, lat, lon) and (date, station) refer to the same data. In this paper we usually model the indexing space as CW-complexes, which are a class of topological spaces built by gluing together n-dimensional balls (which include points, intervals, filled circles, filled spheres, etc.) using continuous attaching maps. The topology can be defined to have a resolution equal to the number of indices in a dataset [35] such that the key (continuity)-value (data) pairing is always preserved.

Base Space Data Type We encapsulate the desired properties of the connectivity as the category $\mathcal{K}$ (Supplement subsection 2.1) to express that these are the properties that are required as part of the implementation of the base space data type. The standard construction to create a category from a topological space takes the objects to be all the neighborhoods, and morphisms to be the inclusion maps [40]. For neighborhoods U_1, U_2, there will be at most one morphism $\hat{\phi} \in \mathrm{Hom}(U_1, U_2)$. We denote the objects as K when the underlying fiber bundle is trivial (Supplement subsection 4.1), which is the case for the bundles discussed in this paper.

3.1.2 Field Data Types: Fiber Space F

Spivak shows that data typing can be expressed in a categorical framework in his fiber bundle formulation of tables in relational databases [45, 46]. We adopt Spivak's definitions of *type*, *schema*, *record*, and *table* because that allows us to use a dimension agnostic named typing system for the dataset fields that is consistent with the abstraction we are using to express the continuity. Broadly, Spivak defines *types* using maps π, π_σ from type domain to data type and field name (e.g. $\pi : \mathbb{R} \to$ `float`, $\pi_\sigma : \mathbb{R} \to$ temperature), *schemas* as maps σ from type name to data type (e.g. $\sigma :$ temperature $\to$ `float`), *records* as maps from name to value ($r :$ `temperature` $\to 152.0°F$) and *tables* as maps from index to record (Supplement section 6):



Definition 3.2. We define the fiber space F as the space of all possible data records $F \coloneqq \{r : C \to \mathscr{U}_\sigma \mid \pi_\sigma(r(C)) = C \text{ for all } C \in C\}$ such that the preimage of a point is the corresponding data type domain $\pi^{-1}(k) = F_k = \mathscr{U}_{\sigma_k}$.

Adopting Spivak's fiber bundle construction of types allows our model to reuse types so long as the field names are distinct and that field values can be accessed by field name, since those are sections on $\mathcal{U}_\sigma$. Furthermore, since domains $\mathscr{U}_\sigma$ of types are a mathematical space, multi-dimensional fields can be encoded in the same manner as single dimensional fields and fields can have different names but the same type.

Fiber Space Data Type As with the base space category $\mathcal{K}$, we propose a fiber category $\mathcal{F}$ to encapsulate the properties of the field types of the data. The fiber category has a single object F of an arbitrary data type, morphisms on the fiber object $\hat{\phi} \in \mathrm{Hom}(F, F)$.

3.1.3 Composition: Adding and Expanding Records

Index spaces K_a and K_b can be composed by taking the fibered coproduct $\oplus : K_a, K_b \mapsto K_a \sqcup_{K_c} K_b$ over a shared space K_c. Fields F_a and F_b can be composed by taking the fiber product $\otimes : F_a, F_b \mapsto F_a \times_{F_c} F_b$ over a shared fiber F_c.

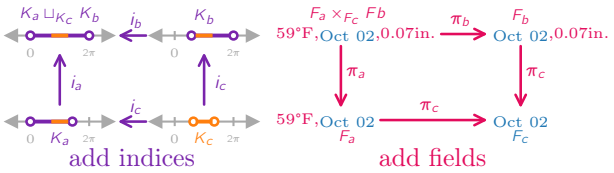


Figure 2: Records are added to the dataset as the *fibered coproduct* $K_a \sqcup_{K_c} K_b$ of index spaces. Fields are added to each record as the *fiber product* $F_a \times_{F_c} F_b$ of fiber spaces.

The fibered coproduct is a generalization of inserting non-duplicated records into a database, while the fiber product is a generalization of a database join on a key. For example, in Figure 2, the neighborhood K_c is present in both K_a and K_b; therefore K_a and K_b are glued together where they overlap at K_c. Two spaces have been combined correctly if every point in the overlap $k \in K_c$ is present in each of the spaces $k \in K_a$ and $k \in K_b$ such that it is present in the joined space $k \in (K_a \sqcup_{K_c} K_b)$. In Figure 2, the (temperature, time, pressure) record separates into (temperature, time) and (pressure, time) records that share the same time. Furthermore this time is the same whether it is obtained from the (temperature, time) or (pressure, time) record. Generally, the test for joining fields is that when a record is present in a fiber $r_c \in \pi_c(F_a), r_c \in \pi_c(F_b)$ then it is in the joint record $r_c \in \pi_c(F_a \times_{F_c} F_b)$ when is in the field F_c being joined on. Verifying that indices are joined correctly allows us to reliably build larger datasets out of smaller ones, such as in the case of distributed and on demand datasets. Verifying that fields are joined together correctly for the same record allows us to reliably combine multiple datasets together on shared properties, e.g. growing the weather station data in Figure 1 from a temporal to spatial dataset by adding the weather at each location at each time.

3.1.4 Data: Section τ

We encode data as a section τ of a bundle because this allows us to incorporate the topology and field types in the data definition. For a neighborhood U in the base space K, a section is a function $\tau : U \to F$. The set of all sections over a specific neighborhood is conventionally denoted as $\Gamma_U(E) \coloneqq \{\tau : U \to E|_U \mid \pi(\tau(k)) = k \text{ for all } k \in U\}$.

Definition 3.3. We define the section τ as the map $\tau : k \mapsto r$ from each point $k_i \in U$ to a record in the fiber space $r \in F_{k_i}$ over k_i.

We can model data as structures that map from an index like point k to a data record r, and encapsulate multiple datasets with the same fiber and base space as different sections of the same bundle. When data exists for every point in the base space (the section is continuous), we can define a global section $\tau : K \to F \in \Gamma_K(E)$ which we translate into a data signature of the form `dataset` : `connectivity`$\to$`field` where $\tau =$ `dataset`, $K =$ `connectivity` and $F =$ `fields`. When the section is not global, we can use the fiber bundle property of local-triviality to define local sections $\tau|_U \in \Gamma_{U_k}(E)$ over a point. Most data sets can be encoded as a collection of local sections $\{\tau|_{U_k}$ for all $k \in K\}$ where each section has the signature `dataset:connectivity` $\to$ `field`.

3.1.5 Example: Uniform Abstract Graphic Representation

One of the reasons we use fiber bundles as an abstraction is that they are general enough that we can also encode the output of a visual algorithm as a bundle. We denote the output as a graphic, but the use of bundles allows us to generalize to output on any

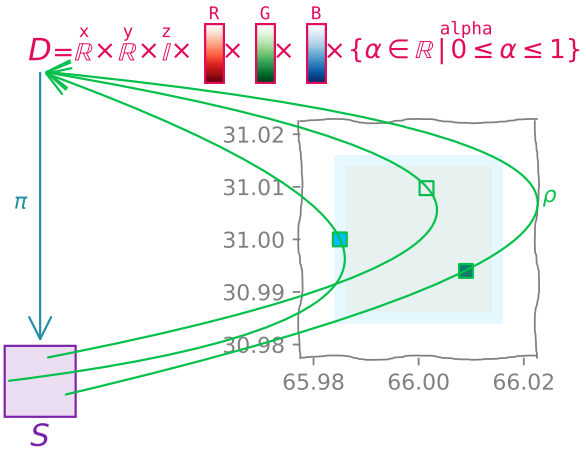


Figure 3: The pixels in the scatter marker are the records returned by the section function ρ over the points $s \in S$.

display space, such as a screen or 3D print. The graphic bundle $D \hookrightarrow H \overset{\pi}{\twoheadrightarrow} S$ consists of the total space H which is an abstraction of an ideal (infinite resolution) space into which the graphic can be rendered, the base space S which is a parameterization of the display area, e.g. the inked bounding box in the cairo [1] rendering library, and the fiber space D which is an abstraction of the renderer fields (e.g. a 2 dimension screen has pixels that can be parameterized $D = \{x, y z, r, g, b, a\}$). As with data, we model the graphic specifications as sections ρ of the graphic bundle $\Gamma_W(H) := \{\rho : W \to H|_W \mid \pi(\rho(s)) = s \text{ for all } s \in W\}$. The sections ρ map from a point in an openset in the graphic space $s \in W \subseteq S$ to a point in the graphic fiber D. The section evaluated on a single point s returns a single graphic record, e.g., one pixel in an ideal resolution space. In our model, the unevaluated graphic section is passed from the visualization library component to the renderer to generate graphics.

In Figure 3, the bundle H is a trivial bundle so $H = D \times S$; therefore, the section function $\rho : S \to D$ maps into the fiber for a simplified 2D RGB infinite resolution pre-render space and returns the $\{x, y, r, g, b\}$ values of a pixel in an infinite resolution space. In Figure 3 these pixels are approximated as the small colored boxes. Each pixel is the output of the $\rho(s)$ section that intersects the box. The set of all pixels returned by a section evaluated on a given visual base space $\rho|_S$ can yield a visual element, such as a marker, line, or piece of a glyph and in Figure 3 is a blue circle with a black edge. While Figure 3 illustrates a highly idealized space with no overlaps, overlaps can be managed via a fiber element D_z for ordering. It is left to the renderer to choose how to blend D_z and D_α layers.

3.1.6 Data transformations

Data is often codified as the set of values and the mathematical actions (Supplement subsection 1.2) on those values, e.g., the Steven's measurement scales define nominal, ordinal, interval, and ratio data by the allowed operations on the data [47]; other researchers have since expanded the scales to encapsulate more types of structure [29, 50]. We separate data transformations into transformations on the base space $(\hat{\phi}, \hat{\phi}^*)$ and transformations on the fiber space $\tilde{\phi}$.

Connectivity The base space transformation $\hat{\phi}$ is a continuous map between two base spaces: $\hat{\phi} : K' \to K$. For any such map between topological spaces, a bundle over K transforms into the pullback bundle over K'. A section $\tau : K \to E$ over K is transformed into a section $\tau' = \tau \circ \hat{\phi}$ through the pullback $\hat{\phi}^*$. This means that the index of a record changes $k = \hat{\phi}(k')$ but the values in the record are unmodified $\tau(k) = \hat{\phi}^* \tau(k') = \tau(\hat{\phi}(k'))$.

Records The fiber space transformation $\tilde{\phi}$ is a continuous map between two base spaces: $\tilde{\phi} : F \to F'$. The function $\tilde{\phi}$ transforms $\tau(k) \in F$ into $\tilde{\phi}(\tau(k)) \in F'$. To be structure preserving, $\tilde{\phi}$ must be a morphism in the category containing the fiber.

scale	operators	sample constraint
nominal	$=, \neq$	$\tau(k_1) \neq \tau(k_2) \implies \tilde{\phi}(\tau(k_1)) \neq \tilde{\phi}(\tau(k_2))$
ordinal	$<, \leqslant, \geqslant, >$	$\tau(k_1) \leqslant \tau(k_2) \implies \tilde{\phi}(\tau(k_1)) \leqslant \tilde{\phi}(\tau(k_2))$
interval	$+, -$	$\tilde{\phi}(\tau(k) + C) = \tilde{\phi}(\tau(k)) + C$
ratio	$*, /$	$\tilde{\phi}(\tau(k) * C) = \tilde{\phi}(\tau(k)) * C$

Table 1: To be structure preserving with respect to a given measurement scale, $\tilde{\phi}$ must commute with the operators of that scale.

Connectivity and Records We define a full data transformation as one that induces both a remapping of the index space and a change in the data values: $\phi : \tau \mapsto \tilde{\phi} \circ \tau \circ \hat{\phi}$. Fiber transformations applied to different factors of the fiber space (as a product data type) compose into simultaneous transformations of the composite fiber, while base space transformations applied to different components of the base space compose into simultaneous transformations of the composite base space. We can define a composite transform where each fiber transform in a collection can be applied to a different fiber field. These function compositions are implemented in the same manner as record compositions, as described in subsubsection 3.1.3.

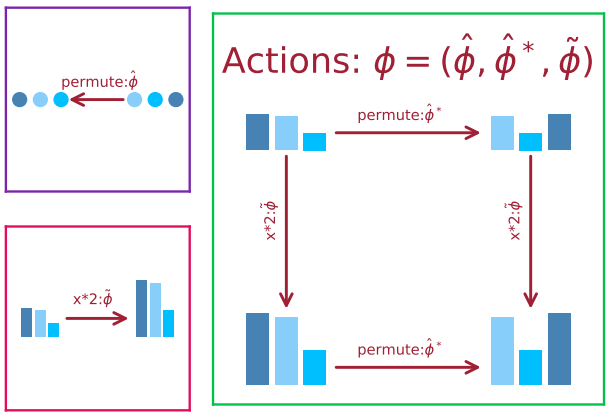


Figure 4: The actions on the base and fiber space commute such that the data can be permuted then scaled or scaled then permuted.

The component actions are independent such that they can be applied in either order. In Figure 4, the action $\hat{\phi}, \hat{\phi}^*$ permutes the indexing space such that locations of the bars relative to each other change. The action $\tilde{\phi}$ doubles the data value associated with the height of the bar. Because these are both structure preserving actions, the order in which the bars are transformed - permute then scale or scale then permute - does not change the final image.

3.2 Abstract Data Containers

While bundles provide a way to describe the structure of the data, sheaves are a mathematical way of describing the data container. Ghrist describes sheaves as an algebraic data structure [14] in that they codify the bookkeeping that data containers must implement to keep track of the continuity of the data. Sheaves and presheaves are

functors (Supplement subsection 2.2), which are mappings between categories that preserve the domains, codomains, composition, and identities of the morphisms within the category [40]. Codifying the data container as the sheaf functor $\mathcal{O}_{K,E} : \mathcal{U} \to \Gamma_{\mathcal{U}}(E)$ expresses that the container is a structure preserving map between the objects in the index space and sets of data records and that the container preserves the maps between index space objects and the maps between data records (supplement). A presheaf (Supplement subsection 5.1) is a functor from an arbitrary category to the set category and a property of presheaves is that they are contravariant, meaning that the morphisms between the input objects $\hat{\phi}$ go in the opposite direction from the morphisms between the output objects $\hat{\phi}^*$, as illustrated in Figure 4. As discussed in subsubsection 3.1.6, $\hat{\phi}$ changes the index and $\hat{\phi}^*$ aligns the records by "pulling back" records to where they belong in the new index. The map between sets is contravariant to the map over indices because not all sets exist over all indices, e.g. when $\hat{\phi}$ is an inclusion mapping that returns a subset of the full index. Sheaves are presheaves that satisfy the gluing and locality axioms (Supplement subsection 5.2). The locality axiom asserts that two sections are equal when they return the same values over the same neighborhood (indices). The gluing axiom says that sections of the dataset defined over subsets of the indexing space can be glued together in a way where the glued together section is equivalent to a section over the whole indexing space. When a data container implements sheaf constraints, the subsets and whole data space are guaranteed to have the same mathematical properties, e.g. morphisms. This means visual algorithms written in our framework treat data that is in memory, distributed, streaming, or on-demand as equivalent and are therefore inherently support representational invariance, as formulated by Kindlemann and Scheidegger [25].

The advantage of using these constraints for specifying structure preserving data constraints is that the test for whether a container implements these constraints is trivial:

axiom	applied to datasets and indexes
presheaf	given $\mathrm{index}_1 \subset \mathrm{index}_2$: $\exists\ \mathrm{dataset}[\mathrm{index}_2]$ $\Rightarrow \mathrm{dataset}[\mathrm{index}_2][\mathrm{index}_1] = \mathrm{dataset}[\mathrm{index}_1]$ $\exists\ \mathrm{dataset}[\mathrm{index}_1] \not\Rightarrow \exists\ \mathrm{dataset}[\mathrm{index}_2]$
locality	$\mathrm{dataset}^1[i] = \mathrm{dataset}^2[i]\ \forall\ i \in \mathrm{index}$ $\Rightarrow \mathrm{dataset}^1 = \mathrm{dataset}^2$
gluing	$i = j$ and $\mathrm{dataset}^1[i] = \mathrm{dataset}^2[j]$ $\mathrm{dataset}^3 \coloneqq \mathrm{dataset}^1[:i] \oplus \mathrm{dataset}^2[j:]$

Table 2: When these axioms are satisfied, we can treat the data container as a sheaf, which means local data availability and consistency constraints are modeled by inclusion maps $\hat{\phi}$ and the resulting restriction maps $\hat{\phi}^*$.

4 Connectivity Preserving Map ξ

From the early maps of stars on, there has been an expectation that the connectivity in visualizations matches the connectivity of the underlying data [12]. This is somewhat codified in the notion of readability [59], which is that visual elements must correspond to distinct data elements. We propose that if connectivity is preserved then there should exist a continuous map ξ that expresses the correspondence between graphic and data indexing spaces.

Definition 4.1. We define the mapping ξ to be a continuous map $\xi : W \to \mathcal{U}$ between a graphic neighborhood $W \subseteq S$ and data neighborhood $\mathcal{U} \subseteq K$.

The set of points in graphic space that correspond to each point in data space is $\xi^{-1}(k) = \{s \mid \xi(s) = k \text{ for all } k \in K, s \in S\}$. We construct the map as going from graphic to data because that encodes the notion that every visual element traces back to the data in some way, but there may be more data available than what is shown in the graphic.

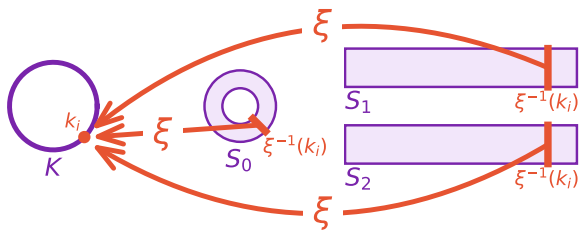


Figure 5: The function ξ maps each point in each orange band on each indexing space S_0, S_1, S_2 to the same point k_i in the circle K.

Each orange band in Figure 5 is defined as $\frac{7\pi}{4} \times [0,1]$ such that each point on each band maps to the same point $\xi|_{\frac{7\pi}{4}\times[0,1]} = \frac{7\pi}{2}$. Despite discarding the cyclical information embedded in K the spaces S_1 and S_2 have compatible structure to their images under ξ as illustrated by the orange bands pulling back to a point.

4.1 Data and Graphic Correspondence

Since we have defined a continuous function ξ between two spaces K, S, we can construct functors that transport sheaves over each space to the other space [18]. This allows us to find what data we expect is being visualized at each graphic index location and what the graphic specification is for the data at each data index location.

	function	constraint
s to k	$\xi : W \to \mathcal{U}$	for $s \in W$ exists $k \in \mathcal{U}$ s.t. $\xi(s) = k$
graphic for k	$\xi_*\rho : \mathcal{U} \to \xi_* H\|_{\mathcal{U}}$	$\xi_*\rho(k)(s) = \rho(s)$
record for s	$\xi^*\tau : W \to \xi^* E\|_W$	$\xi^*\tau(s) = \tau(\xi(s)) = \tau(k)$

Table 3: Correspondences between data and graphic specification. See Supplement subsection 5.3

The parameterization of $\xi_*\rho$ in Figure 6 is intended as an approximation and is akin to declarative visualization specs such as vega [41] and svg [39]. These specs and $\xi_*\rho$ also provide a renderer independent way of describing the graphic and are therefore useful for standardizing internal representation of the graphic and serializing the graphic for portability. As illustrated in Figure 6, ξ^* is critical to interactive techniques such as brushing, linking, and tooltips [4] because they depend on being able to look up which data values go to which parts of the graphic.

4.2 Example: Readable Visualizations

As illustrated in Figure 6, modeling the data and graphic containers as sheaves and constructing structure preserving maps between the indices provides a method of precisely describing the which data values belong with which graphical representations. Definition 4.1 establishes that there is a correspondence between data at an index k_i and graphics generated at S_i. In Figure 6, one example of this correspondence is the the orange bands on S_0, S_1, S_2 that correspond to the orange point k_i. Because of this correspondence and the properties of sheaves, we can construct a graphic specifications for each data index $\xi_*\rho$; in Figure 6 $\xi_*\rho(K_i)$ a svg-like spec describing the piece of the circle, sin, and cosine curves generated for the data at the point k. We can also retrieve the data $\xi^*\tau_{\xi^{-1}(k)}$ that is being visualized by each specific part of the graphic, which in Figure 6 is the section $\tau(k_i)$.

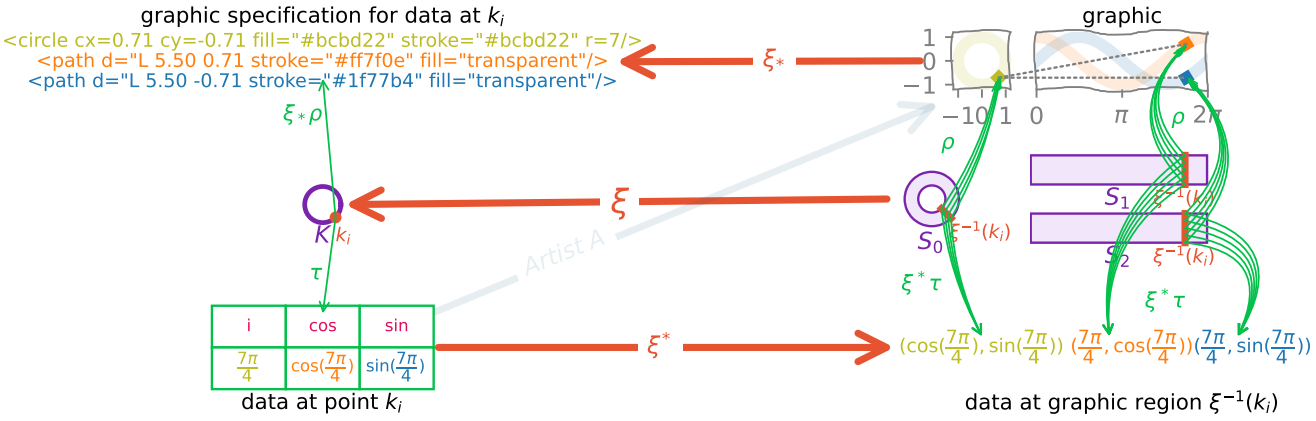


Figure 6: The data τ consists of the $\sin$ and $\cos$ functions over a unit circle base space K visualized as the circle and two line plots generated from the graphic specification ρ. The index map ξ keeps track of which part of the circle, sin, and cosine plots correspond to which point on the unit circle. The pushforward ξ_* matches each point in the data space to the corresponding graphic specification, and the pullback ξ^* matches each point in the graphic space to the data at that point.

5 Artist: Structure Preserving Maps

We build off of Kindlmann and Scheidegger's notion of equivariant [25] and Mackinlay's notion of homomorphic [31] visualizations in our definition of structure preservation.

Definition 5.1. We define a mapping from data to graphic to be **structure preserving** when connectivity is preserved, the structure on the data and the graphics is compatible, and changes to data and graphic commute.

By mapping from data to graphic we mean, formally, a sheaf morphism from the sheaf of local sections on the data fiber bundle to the sheaf of local sections on the graphic fiber bundle: $f : \Gamma_K(E) \to \Gamma_S(H)$. We call the structure preserving mappings Artists[3]:

$$A : \Gamma_K(E) \to \mathrm{Im}_A(S, H) \subset \Gamma_S(H)$$

The output of an artist A is a restricted subset of graphic sections $\mathrm{Im}_A(S, H)$ that are, by definition, only reachable through a structure preserving artist: $\mathrm{Im}_A(S, H) := \{\rho \mid \exists\, \tau \in \Gamma_K(E) \text{ s.t. } A(\tau) = \rho\}$. We define this subset because the space of all sections $\Gamma_S(H)$ is every possible way of setting pixels on a graphic base space, including non structure preserving settings such as setting all the pixels the same color.

A structurally preserving Artist is a *natural transform* (Supplement subsection 2.3), which is a structure preserving map of functors [34]. Through the application of the transport functors (ξ_*, ξ^*), artists can be constructed such that an artist over data space $A_K : \tau \mapsto \xi_*\rho$, an artist over graphic space $A_S : \xi^*\tau \mapsto \rho$, and an artist $A : \tau \mapsto \rho$ are equivalent (Supplement subsection 5.4); therefore, the artist signature can be formulated as:

$$\begin{aligned}\texttt{Artist} : &(\texttt{dataset} : \texttt{connectivity} \to \texttt{fields}) \\ \to &(\texttt{graphic} : \texttt{connectivity} \to \texttt{fields})\end{aligned}$$

where `connectivity` and `field` are meta classes of compatible types. Defining the artist as a *natural transform* means that the Artist must preserve the constraints on the input sheaves and output sections, meaning that actions ϕ on data and graphic must be equivariant (subsection 5.1), and the Artist must respect the sheaf constraints of *locality* and *gluing* for the input and equivalent output. The natural transform property also gives developers the flexibility to connect components in data space with components in graphic space, e.g. using the output of a component that assigns line color in data space to as the input of a line building component that dynamically resamples what is on screen in graphic space.

[3] Artists are named for the Artist class in Matplotlib [23] because the Artist is the parent class of all the objects that assemble and manage visual elements such as points, lines, and shapes.

5.1 Equivariance

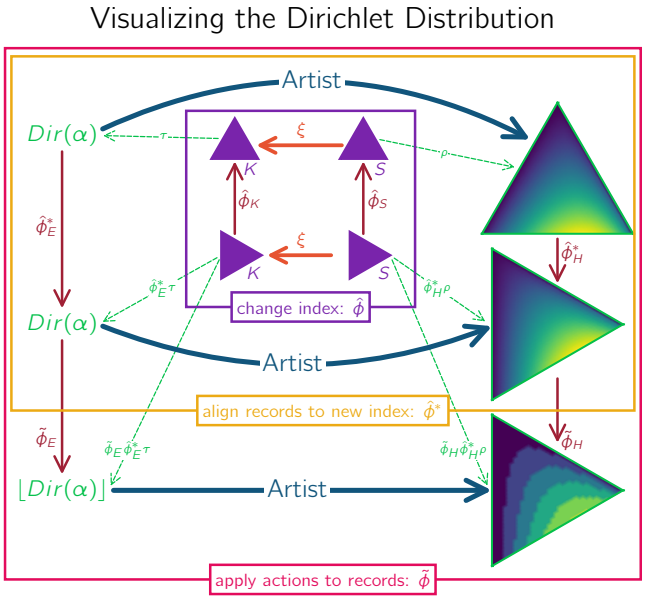


Figure 7: This artist maps a numerical section of the Dirichlet distribution to a corresponding graphical representation. The artist preserves connectivity by not breaking presheaf implementations of rotation $(\hat{\Phi}, \hat{\Phi}^*)$ of the indexing space K, S and field structure by implementing the data to color mapping such that binning $(\hat{\phi})$ the data and mapping the bins to colors is equivalent to binning the colors directly. The subscripts denote the space the action occurs in.

An artist is equivariant when changes to the input and output commute such that for all points $s' \in S'$, starting at an arbitrary data point $\tau(k)$ and transforming it into a different data point and then into a graphic $A(\tilde{\phi}_E(\tau(\hat{\phi}_E(\xi(s')))))) = \tilde{\phi}_H(A(\tau(\xi(\hat{\phi}_H(s'))))))$ is equivalent to transforming the original data point into a graphic and then transforming the graphic into another graphic. The function $\hat{\phi}_H$ induces a change in graphic generating function that matches the change in data. For example, Figure 7 shows an artist function A that maps a Dirichlet distribution on a triangular base space to a corresponding graphic. The artist is equivariant with respect to connectivity because the base space rotation actions $\hat{\phi}, \hat{\phi}^*$ commute such that rotating the data and then mapping it to graphic or rotating the graphic yield the same Dirichlet distribution graphic where the values have moved to new locations but not changed. The artist is also equivariant with respect to field structure because the binning action $\tilde{\phi}$ commutes such that binning and then color mapping the data yields an equivalent graph to binning the colormap directly.

The graphic transformation $\hat{\phi}_H$ is difficult to define because by definition it acts on a single record, e.g. a pixel in an idealized 2D screen. Instead, we describe the changes on the graphic side as changes in measurements M of visual components. A superset of the Bertin visual variables [5], measurable components are attributes of the rendered graphic that can be quantified, such as the color, position, shape, texture, or rotation angle of the graphic. For example, a measurement of a scatter marker could be its color (e.g. red) or its x position (e.g. 5). The measurable elements can only be computed over the entire preimage because these aspects, such as thickness or marker shape, refer to the entire visual element.

$$\begin{array}{ccc} \Gamma_K(E) & \xrightarrow{A} & \mathrm{Im}_A(S,H) \\ \eta\downarrow & \overset{\delta}{\swarrow} & \downarrow \text{render} \\ \mathrm{Hom}(K,M) & \xleftarrow[\text{measure}]{} & \text{visualization} \end{array}$$

We define an output verification function $\delta : (\rho \circ \xi^{-1}) \mapsto (K \xrightarrow{\delta_\rho} M)$ that takes as input the section evaluated on all the graphic space associated with a point $\rho_{\xi^{-1}|_k}$ and returns the corresponding measurable visual components M_k. The verification function is equivalent to measuring components of the rendered image $\delta = \text{measure} \circ \text{render}$, which means an alternative way of implementing the function when S is not accessible is by decomposing the output into its measurable components. We also introduce a function $\eta : \tau \mapsto (K \xrightarrow{\eta_\tau} M)$ that maps data to the measurement space directly such that $\eta_\tau(k)$ is the expected set of measurements M_k. The pair of verification functions (η, δ) can be used to test that the expected encoding η_τ of the data matches the actual encoding δ_ρ: $\eta(\tau)(k) = \delta(A(\tau))(k) = \delta(\rho \circ \xi^{-1})(k) = M_k$

As introduced in subsubsection 3.1.6, the base space transformation $\hat{\phi}$ is invariant because $\tau|_U = \tau|_{\hat{\phi}(U')}$. This means that, for all points in the data $k \in K$, the measurement should not change if only the base space is transformed $\eta(\tau)(\hat{\phi}(k')) = \delta(A(\tau))(k)$. In contrast, a change in sections induces an equivalent change in measurements $\eta(\tilde{\phi}(\tau))(k) = \tilde{\phi}_M(\delta(A(\tau))(k))$. The change in measurements $\tilde{\phi}_M$ is defined by the developer as the symmetry between data and graphic that the artist is expected to preserve.

5.2 Composing Artists

As shown in Figure 8, many graphics are compositions of smaller graphics, here a scatter plot and choropleth. Artists are composable because they preserve data and graphic composition subsubsection 3.1.3 and so must preserve the property of the coproduct that $\mathrm{Hom}(K_1 \sqcup K_2, F) \simeq \mathrm{Hom}(K_1, F) \times \mathrm{Hom}(K_2, F)$ and the property of the product that $\mathrm{Hom}(K, F_1 \times F_2) = \mathrm{Hom}(K, F_1) \times \mathrm{Hom}(K, F_2)$ [30], which we generalize as $\mathrm{Hom}(\coprod_i K_i, \prod_j F_j) = \coprod_i \prod_j \mathrm{Hom}(K_i, F_j)$ when composing

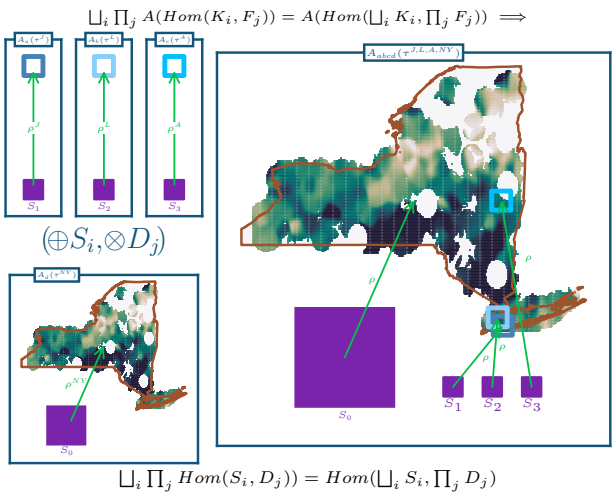


Figure 8: The scatter plot of stations on the rain map from Figure 1 can either be generated as a composition of scatter and map drawing functions or a singular scatter and map drawing function.

with section functions $\tau \in \mathrm{Hom}(K, F)$ to output graphic sections $\rho \in \mathrm{Hom}(S, D)$. We use Hom notation here because it allows us to clearly express how composition with the artist differs for each space $\coprod_i \prod_j A(\mathrm{Hom}(K_i, F_j)) = A(\mathrm{Hom}(\coprod_i K_i, \prod_j F_j)) \implies \coprod_i \prod_j \mathrm{Hom}(S_i, D_j) = \mathrm{Hom}(\coprod_i S_i, \prod_j D_j)$ such that the same graphic can be generated from a singular artists on the entire data set. When combining spaces $\oplus : K_a, K_b$, we can construct the graphics section as the piecewise function

$$\rho(s) \coloneqq \begin{cases} \rho^a(s) & s \in \xi^{-1}(K^a) \\ \rho^b(s) & s \in \xi^{-1}(K^b) \end{cases}$$

When combining fiber spaces $\otimes : F_a, F_b$, we construct the graphics function as the set of sections $\rho^{a,b} = \{\rho^a(s), \rho^b(s)\}, s \in \xi^{-1}(K)$ but in the single output case, such as a figure to a screen, the output fiber is identical $D_1 \times D_2 = D$ such that we can default to the piecewise construction.

We can verify that base space composition was implemented correctly by verifying that the artist and composition of artists return the same graphic on points in the intersection K^c. Given $k_a \in K_c \subset K_a$ and $k_b \in K_c \subset K_b$, if $k_a = k_b$ then $A_{ab}(\tau^{ab}(k_a)) = A_a(\tau^a(k_a)) = A_b(\tau^b(k_b))$ for all $k_a, k_b \in K_a \bigsqcup_{K_c} K_b$. This tests whether graphics line up when drawn in parts, e.g. algorithms for visualizing streaming data [26] and techniques that rely on shifting or resampling the indexing space, such as panning and zooming [37]. We can verify that fiber space composition was implemented correctly by checking that any shared fiber F^c is encoded consistently $\delta(A_{ab}(\tau^{ab}(k)))|_{F^c} = \delta(A_a(\tau^a(k_a)))|_{F^c} = \delta(A_b(\tau^b(k_b)))|_{F^c}$. This expectation of using the same encoding for the same variable is a generalization of the concept of consistency checking of multiple view encodings discussed by Qu and Hullman [38]. In Figure 8, this is a check that, e.g., the scatter marker drawn by A_a is identical to the scatter marker drawn by A_{abcd}.

5.2.1 Animation and Interactivity

Animation and interaction can be thought of as either composition of base spaces (more frames), fiber spaces (additions to the graphic

such as annotations, or cursors), or a combination therefore; therefore structure preserving animations and interactions fall out as a consequence of implementing structure preserving artists such that it is structure preserving for all frames $\{A(\tau(k)) \mid k \in K\}$. Panning, zooming, and scrolling rely on sheaf constraints that ensure that continuity is maintained as the data is subsetted, selection and hover require the corresponding information provided by the pushforward ξ_* and pullback ξ^* functors, and brushing, linking, and annotation rely on the bookkeeping of ξ and effectively implemented composition to ensure that the pieces in graphic are in sync such that they all map back to the same subset of the data or data point.

6 Constructing Structure Preserving Components

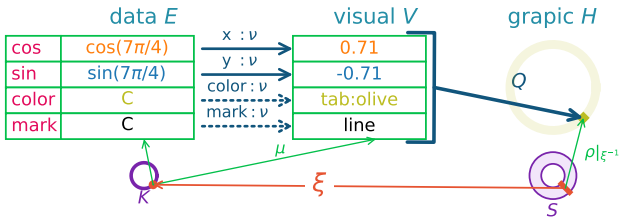


Figure 9: The artist function that draws the green point on the circle is composed of two stages $A = \nu \circ Q$. The encoder ν maps the data values to visual values and the compositor Q generates the graphic specification for producing the sections of the circle. Default encodings, such as color and mark type, are here treated as pullbacks of constant values in the data.

Artists naturally factorize into a two-stage process, starting with an encoding stage ν followed by a compositing stage Q. In the encoding stage ν, a data bundle is treated as separable fields and each field is mapped to a measurable visual variable. In the encoding stage, many of the expected visual mappings η can be implemented inside the library. Factoring out the encoding stage leaves the compositing stage Q responsible for faithfully translating those measurable visual components into a visual element.

6.1 Indexing Space

We can use the data base space as an indexer into the graphic base space if we implement the graphic base space as if there was a deformation retraction (Supplement subsection 3.4) from the graphic space to the data space:

$$\underbrace{K \times [0,1]^n}_{S} \xrightarrow{\xi} K, \; n = \begin{cases} \dim(S) - \dim(K) & \dim(K) < \dim(S) \\ 0 & \text{otherwise} \end{cases}$$

where n is a thickening of the graphic base space S to account for the dimensionality of the output space because otherwise the data dimensionality K may be too small for a graphic representation. For example, a line is 1D but is a 2D glyph on a screen; therefore the graphic space S is constructed by multiplying the base space K with an interval $[0,1]$.

6.2 Measurable Visual Components

We encapsulate the space of measurable components reachable through the encoding stage ν as a visual fiber bundle $P \hookrightarrow V \xrightarrow{\pi} K$. The restricted fiber space P of the bundle acts as the specification of the internal library representation of the measurable visual components. The space of visual sections $\Gamma_U(V) \coloneqq \{\mu : U \to V|_U \mid \pi(\mu(k)) = k \text{ for all } k \in U\}$ return a visual encoding $\mu(k)$ corresponding to data record $\tau(k)$. Since the data bundle E and visual bundle V have the same continuity $\pi(\tau(k)) = \pi(\mu(k))$, they are considered structurally equivalent such that $E = V$. The distinguishing characteristic of V is that it is part of the construction of the artist and therefore a part of the visualization library implementation. We propose that reusing the fibers P across components facilitates standardizing internal types across the library and that this standardization improves maintainability.

6.3 Component Encoders

As introduced in subsection 5.1, there is a set η of functions that map between data and corresponding visual encodings. We propose that for visualization library components to be structure preserving, they must implement a constrained subset of these encoding functions $\Gamma_K(E) \xrightarrow{\nu} \Gamma_K)V) \subset \Gamma_K(E) \xrightarrow{\eta} \mathrm{Hom}(K, M)$ that preserve the categorical structure (operators and morphisms) of the fiber and the continuity of the data section. As mentioned in subsection 6.2, the total visual space is restricted to the space of data types internal to the library $P \subset M$ and sections are subsets of homsets $\Gamma(K, V) \subset \mathrm{Hom}(K, M)$ because sections must be continuous. The encoding functions ν are fiber wise transforms such that $\pi(E) = \pi(\nu(E))$. A consequence of this property is that ν can be constructed as a point wise transformation $\nu : F_k \to P_k$, which means that means that a point in a single data fiber $r \in F_k$ can be mapped into a corresponding point in a visual fiber P_k. This means that an encoding function ν can convert a single record independent of the whole dataset.

Since E and V are structurally identical, any V can be redefined as E; therefore any collection of ν functions can be composed such that they are equivalent to a ν that directly converts the input to the output:

$$F_k \xrightarrow{\nu} P_k \coloneqq F'_k \xrightarrow{\nu'} P'_k, \quad \nu'' \text{ (dashed)}$$

Because the encoder ν is a natural transform between data and visual sheaves, ν can be composed in the same manner as artists as described in subsection 5.2.

6.4 Graphic Compositor

The compositor function Q transforms the measurable components into properties of a visual element. The compositing function $Q : \Gamma_K(V) \to \Gamma_S(H)$ transforms the sections of visual elements μ into sections of graphics ρ. The compositing function is a map from sheaves over K to sheaves over S because the graphic section must be evaluated on all points in the graphic space to generate the visual element corresponding to a data record at a single point $A(\tau(k)) = \rho(\xi^{-1}(k))$. Because the compositor Q is a natural transform between visual and graphic sheaves, Q can be composed in the same manner as artists as described in subsection 5.2.

Since encoder functions are infinitely composable, as described in subsection 6.3, a new compositor function Q can be constructed by pre-composing ν functions with the existing Q:

$$\Gamma_K(V) \xrightarrow{\nu} \Gamma_K(V') \xrightarrow{Q} \Gamma_S(H), \quad Q' \text{ (dashed)}$$

The composition in subsection 6.4 means that different measurable components can yield the same visual elements. As shown in Figure 9, a set of ν functions individually convert the values in the data record to visual components. Then the Q function combines these visual encodings to produce a graphic section ρ. When this section is evaluated on the graphic space associated with the data

$\rho(\xi^{-1}(k))$, it produces green pixels at the intersection of the x and y positions listed in μ. The composition rule in subsection 6.4 means that developers can implement Q as drawing a circle or can implement a Q that draws arbitrary shapes, and then provide different ν adapters, such as one that specifies that the shape is a circle.

6.5 Verification

The factorization of the artist into the encoding and composition stages provides a framework for testing the conversion from data to visual component separately from the assembly of components into a glyph. For example, encoders built in this framework could be designed to ensure that a transformation from a non-colorblind safe colormap into a colorblind safe colormap would ensure that the colormaps are equivalent such that the transformation is not introducing a change in structure. Similarly, the composition stage can be tested to ensure that parameters are applied consistently across visual elements and, as with encoding, more accessible elements can be swapped in without introducing a change in structure.

6.5.1 Encoder

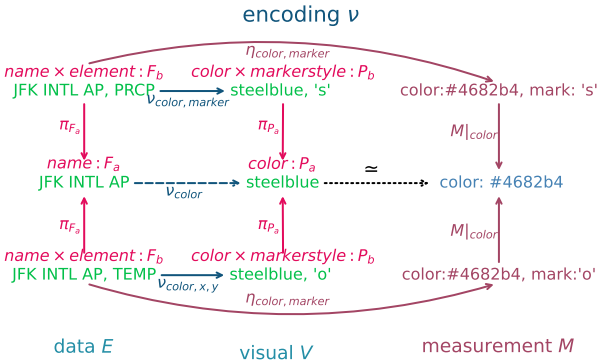


Figure 10: The encoders $\nu_{color,marker}$ and ν_{color} correctly encode JFK INTL AP as steelblue if they return steelblue and it matches the reference color #4682b4. An external testing library can follow a similar procedure using η functions to test library encoders ν.

An encoding function ν is equivariant if the change in data, as defined in subsubsection 3.1.6, and change in visual components are equivariant. Since E and V are over the same base space and are point wise, the base space change $\hat{\phi}_E$ applies to both sides of the equation $\nu(\tau_E(\hat{\phi}_K(k'))) = \mu(\hat{\phi}_K(k'))$ and therefore there should not be a change in encoding. On the other hand, a change in the data values $\tilde{\phi}_E$ must have an equivalent change in visual components $\tilde{\phi}_V\nu(\tau(k)) = \nu(\tilde{\phi}_E(\tau(k)))$ The change in visual components $\tilde{\phi}_V$ is dependent both on $\tilde{\phi}_E$ and the choice of visual encoding.

6.5.2 Compositor

An advantage of factoring out encoding and verification, as discussed in subsubsection 6.5.1, is that the responsibility of the compositor can be scoped to translating measurable components into visual elements. One way of verifying that a compositor is consistent is by verifying that it passes through one encoding even while changing others. For example, when $Q_{ab} = Q_{ac}$ then the output should differ in the same measurable components as μ_{ab} and μ_{ac}. A compositor function Q is equivariant if the renderer output changes in a way equivariant to the data transformation. This means that a change in base space $\hat{\phi}_E$ should have an equivalent change in visual element base space, which means that there should be no change in visual measurement $\mu(\hat{\phi}_K(k')) =$

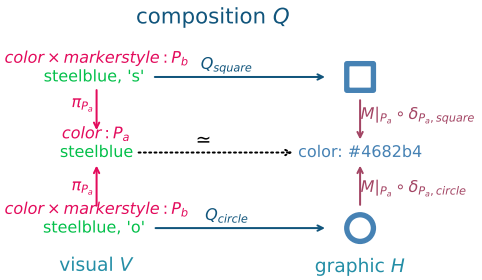


Figure 11: The compositors Q_{square} amd Q_{circle} correctly compose the visual parameter "steelblue" in their outputs if the measured color is the same in the output of both compositors is equal to the reference color #4682b4.

$\delta(Q(\mu)(\hat{\phi}_K(\xi^{-1}k))) = M_k$. As discussed in subsection 5.1, the change in base space may induce a change in locations of measurements relative to each other in the output; this can be verified via checking that all the measurements have not changed relative to the original positions $M_k = M_{k'}$ and through separate measurable variables that encode holistic data properties, such as orientation or origin. The compositor function is also expected to be equivariant with respect to changes in data and measurable components $\tilde{\phi}_V(\mu(k)) = \tilde{\phi}_M(Q(\mu(k)))$ which means that any change to a measurable component input must have a measurably equivalent change in the output. As illustrated in Figure 7, the compositor Q is expected to assemble the measurable components such that base space changes, e.g., reflection, are reflected in the output; faithfully pass through equivariant measurable components, such as binned colors; and ensure that both types of transformations, here scaling and transposition, are present in the final glyph.

7 Conclusion

This paper demonstrates a mathematical formalism for representing the architecture of components in a data visualization library, in which the mathematical structures both enforce well known principles of data visualization and suggest factorizations and decompositions the result in highly modular infrastructure, such that artists can be constructed from composable primitive. These implementations aim to lower maintenance burden because, by definition, they are designed in conjunction with tests that verify that they are structure preserving. There has been ongoing work to implement a new data container and visualization pipeline [28] in Matplotlib that aims to provide a uniform data interface that does on demand querying (i.e. evaluate section of the data); this will allow building in support for functional and on demand data, more uniform handling of statistical calculations, and simplifying how animation works. The new visualization pipeline builds a graph of encoders and a compositor, thereby factoring out the artist specific encodings that cause inconsistencies in how encodings are applied. This new pipeline is also more explicit about how visual elements correspond to distinct data elements, which would allow the artist to retain more information about the visual elements and this information could be passed along to accessibility tools. Unifying the model of internal state could provide a consistent interface for accessibility tools to query for information about the state of the visualizations and for developers writing new functionality to hook into. As the examples with Matplotlib demonstrate, this framework can be used to build components that are compatible with the existing library architecture, allowing for a slow iterative transition to components built using this framework that aligns with maintenance priorities.

Supplemental Materials

See the attached supplement for detailed explanations of the math used in this paper.

Acknowledgments

The authors would like to thank the anonymous reviewers who gave feedback on various versions of this paper. The authors are also grateful to the various Matplotlib and Napari contributors, particularly Juan Nunez-Iglesias, and Nicolas Kruchten for their valuable feedback. Hannah is also very grateful to Nicolas for the suggestion of augmented notation, to the nlab and wikipedia contributors who wrote clear explanations of many of the topics discussed in this paper, to Noa Tamir and Melissa Weber Mendonça for their support, and to Kyle Sunden for his colloboration. This project has been made possible in part by grant number 2019-207333 (EOSS Cycle 1) and EOSS Cycle 3 Chan Zuckerberg Initiative DAF, an advised fund of Silicon Valley Community Foundation.

References

[1] Cairographics.org. 4

[2] A. Bartonicek, S. Urbanek, and P. Murrell. No More, No Less than Sum of Its Parts: Groups, Monoids, and the Algebra of Graphics, Statistics, and Interaction. 34(3):1063–1074. doi: 10.1080/10618600.2024.2429708 1

[3] M. Bastian, S. Heymann, and M. Jacomy. Gephi: An open source software for exploring and manipulating networks. vol. 3 of *Proceedings of the International AAAI Conference on Web and Social Media*. 2

[4] R. A. Becker and W. S. Cleveland. Brushing Scatterplots. 29(2):127–142. doi: 10.1080/00401706.1987.10488204 5

[5] J. Bertin. II. The Properties of the graphic system. In *Semiology of Graphics*. ESRI Press. 7

[6] J. Bertin. *Semiology of Graphics : Diagrams, Networks, Maps*. ESRI Press. 1

[7] Bokeh Development Team. *Bokeh: Python Library for Interactive Visualization*. 1

[8] M. Bostock, V. Ogievetsky, and J. Heer. D³ Data-Driven Documents. 17(12):2301–2309. doi: 10.1109/TVCG.2011.185 1

[9] D. M. Butler and S. Bryson. Vector-Bundle Classes form Powerful Tool for Scientific Visualization. 6(6):576. doi: 10.1063/1.4823118 2

[10] D. M. Butler and M. H. Pendley. A visualization model based on the mathematics of fiber bundles. 3(5):45. doi: 10.1063/1.168345 2

[11] E. H. Chi. A taxonomy of visualization techniques using the data state reference model. In *IEEE Symposium on Information Visualization 2000. INFOVIS 2000. Proceedings*, pp. 69–75. doi: 10.1109/INFVIS.2000.885092 1

[12] M. Friendly. A Brief History of Data Visualization. In *Handbook of Data Visualization*, pp. 15–56. Springer Berlin Heidelberg. doi: 10.1007/978-3-540-33037-0_2 5

[13] B. Geveci, W. Schroeder, A. Brown, and G. Wilson. VTK. 1:387–402. 1

[14] R. W. Ghrist. *Elementary Applied Topology*, vol. 1. Createspace Seattle. 2, 4

[15] A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using NetworkX. In G. Varoquaux, T. Vaught, and J. Millman, eds., *Proceedings of the 7th Python in Science Conference*, pp. 11–15.

[16] P. Hanrahan. VizQL: A language for query, analysis and visualization. In *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data*, SIGMOD '06, p. 721. Association for Computing Machinery. doi: 10.1145/1142473.1142560 2

[17] M. D. Hanwell, K. M. Martin, A. Chaudhary, and L. S. Avila. The Visualization Toolkit (VTK): Rewriting the rendering code for modern graphics cards. 1–2:9–12. doi: 10.1016/j.softx.2015.04.001 1

[18] G. Harder and K. Diederich. *Lectures on Algebraic Geometry I: Sheaves, Cohomology of Sheaves, and Applications to Riemann Surfaces*. Aspects of Mathematics. Vieweg+Teubner Verlag. 5

[19] A. Head, A. Xie, and M. A. Hearst. Math augmentation: How authors enhance the readability of formulas using novel visual design practices. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, CHI '22, article no. 491. Association for Computing Machinery. doi: 10.1145/3491102.3501932 2

[20] J. Heer and M. Agrawala. Software design patterns for information visualization. 12(5):853–860. doi: 10.1109/TVCG.2006.178 2

[21] Z. Hu, J. Hughes, and M. Wang. How functional programming mattered. 2(3):349–370. doi: 10.1093/nsr/nwv042 2

[22] J. Hughes. Why Functional Programming Matters. 32(2):98–107. doi: 10.1093/comjnl/32.2.98 2

[23] J. Hunter and M. Droettboom. The Architecture of Open Source Applications (Volume 2): Matplotlib. 6

[24] J. D. Hunter. Matplotlib: A 2D graphics environment. 9(3):90–95. doi: 10.1109/MCSE.2007.55 1

[25] G. Kindlmann and C. Scheidegger. An Algebraic Process for Visualization Design. 20(12):2181–2190. doi: 10.1109/TVCG.2014.2346325 1, 5, 6

[26] M. Krstajić and D. A. Keim. Visualization of streaming data: Observing change and context in information visualization techniques. In *2013 IEEE International Conference on Big Data*, pp. 41–47. doi: 10.1109/BigData.2013.6691713 7

[27] N. Kruchten, A. M. McNutt, and M. J. McGuffin. Metrics-Based Evaluation and Comparison of Visualization Notations. 30(1):425–435. doi: 10.1109/TVCG.2023.3326907 2

[28] Kyle Sunden. Proof of Concept: Data Containers by ksunden · Pull Request #30865 · matplotlib/matplotlib. 9

[29] W. A. Lea. A formalization of measurement scale forms. 1(1):81–105. doi: 10.1080/0022250X.1971.9989789 4

[30] S. Mac Lane. *Categories for the Working Mathematician*. Graduate Texts in Mathematics. Springer. doi: 10.1007/978-1-4757-4721-8 7

[31] J. Mackinlay. Automating the design of graphical presentations of relational information. 5(2):110–141. doi: 10.1145/22949.22950 1, 2, 6

[32] J. Mackinlay, P. Hanrahan, and C. Stolte. Show me: Automatic presentation for visual analysis. 13(6):1137–1144. doi: 10.1109/TVCG.2007.70594 2

[33] J. D. Mackinlay. Automatic design of graphical presentations. 1

[34] B. Milewski. *Category Theory for Programmers*. Bartosz Milewski. 6

[35] J. R. Munkres. *Elements of Algebraic Topology*. Addison-Wesley. 3

[36] T. Munzner. What: Data Abstraction. In *Visualization Analysis and Design*, AK Peters Visualization Series, pp. 20–40. A K Peters/CRC Press. doi: 10.1201/b17511-3 2

[37] D. Nekrasovski, A. Bodnar, J. McGrenere, F. Guimbretière, and T. Munzner. An evaluation of pan & zoom and rubber sheet navigation with and without an overview. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '06, pp. 11–20. Association for Computing Machinery. doi: 10.1145/1124772.1124775 7

[38] Z. Qu and J. Hullman. Keeping multiple views consistent: Constraints, validations, and exceptions in visualization authoring. 24(1):468–477. doi: 10.1109/TVCG.2017.2744198 7

[39] A. Quint. Scalable vector graphics. 10(3):99–102. doi: 10.1109/MMUL.2003.1218261 5

[40] E. Riehl. *Category Theory in Context*. Dover Publications. 3, 5

[41] A. Satyanarayan, K. Wongsuphasawat, and J. Heer. Declarative interaction design for data visualization. In *Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology*, pp. 669–678. ACM. doi: 10.1145/2642918.2647360 2, 5

[42] C. A. Schneider, W. S. Rasband, and K. W. Eliceiri. NIH Image to ImageJ: 25 years of image analysis. 9(7):671–675. doi: 10.1038/nmeth.2089 2

[43] N. Sofroniew, T. Lambert, K. Evans, P. Winston, J. Nunez-Iglesias, G. Bokota, K. Yamauchi, A. C. Solak, ziyangczi, G. Buckley, M. Bussonnier, D. D. Pop, T. Tung, V. Hilsenstein, Hector, J. Freeman, P. Boone, alisterburt, A. R. Lowe, C. Gohlke, L. Royer, H. Har-Gil, M. Kittisopikul, S. Axelrod, kir0ul, A. Patil, A. McGovern, A. Rokem, Bryant, and G. Peña-Castellanos. Napari/napari: 0.4.5rc1. doi: 10.5281/zenodo.4533308 2

[44] E. Spanier. *Algebraic Topology*. McGraw-Hill Series in Higher Mathematics. Springer. 2
[45] D. I. Spivak. Databases are categories. 2, 3
[46] D. I. Spivak. Simplicial databases. doi: 10.48550/ARXIV.0904.2012 2, 3
[47] S. S. Stevens. On the Theory of Scales of Measurement. 103(2684):677–680. 1, 4
[48] C. Stolte, D. Tang, and P. Hanrahan. Polaris: A system for query, analysis, and visualization of multidimensional relational databases. 8(1):52–65. doi: 10.1109/2945.981851 2
[49] S. Studies. Culturevis/imageplot. 2
[50] M. A. Thomas. Mathematization, Not Measurement: A Critique of Stevens' Scales of Measurement. doi: 10.2139/ssrn.2412765 4
[51] M. Tory and T. Moller. Rethinking visualization: A high-level taxonomy. In *IEEE Symposium on Information Visualization*, pp. 151–158. doi: 10.1109/INFVIS.2004.59 1
[52] P. Vickers, J. Faith, and N. Rossiter. Understanding Visualization: A Formal Approach Using Category Theory and Semiotics. 19(6):1048–1061. doi: 10.1109/TVCG.2012.294 1
[53] H. Wickham. *Ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. 2
[54] V. Wiels and S. Easterbrook. Management of evolving specifications using category theory. In *Proceedings 13th IEEE International Conference on Automated Software Engineering (Cat. No.98EX239)*, pp. 12–21. doi: 10.1109/ASE.1998.732561 2
[55] L. Wilkinson. *The Grammar of Graphics*. Statistics and Computing. Springer-Verlag New York, Inc., 2nd ed ed. 1, 2
[56] K. Wongsuphasawat. Navigating the Wide World of Data Visualization Libraries. 1
[57] K. Wongsuphasawat. Navigating the Wide World of Data Visualization Libraries (on the web). 1
[58] B. A. Yorgey. Monoids: Theme and variations (functional pearl). 47(12):105–116. doi: 10.1145/2430532.2364520 2
[59] C. Ziemkiewicz and R. Kosara. Embedding Information Visualization within Visual Representation. In Z. W. Ras and W. Ribarsky, eds., *Advances in Information and Intelligent Systems*, pp. 307–326. Springer Berlin Heidelberg. doi: 10.1007/978-3-642-04141-9_15 1, 5